

Triana — Documentación técnica del backend

Clases, módulos y funciones principales de lo que llevamos construido

Generado el 09/07/2026 · Carpeta: backend/

Arquitectura general

```
index.js (punto de entrada)
├─ src/database/index.js      → Database      (Supabase: clientes, cuotas, pagos, storage)
├─ src/whatsapp/index.js      → WhatsAppService (whatsapp-web.js: conexión y mensajes)
├─ src/services/ia.service.js  → IAService      (OpenAI: intención, respuestas, OCR→datos)
├─ src/services/ocr.service.js → OCRService     (Tesseract: imagen→texto)
├─ src/services/pagos.service.js → PagosService  (orquesta comprobante→pago→informe)
├─ src/jobs/cron.jobs.js      → CronJobs       (recordatorios e informe diario)
├─ src/routes/index.js        → API REST       (registrar venta, dashboard)
└─ src/utils/telefono.js      → normalizarTelefono (formato consistente de números)
```

index.js punto de entrada

Arranca todo el sistema en orden: conecta la base de datos, levanta la API, inicializa WhatsApp y activa los cron jobs. Si algo falla en el arranque, corta el proceso.

FUNCIÓN	QUÉ HACE
<code>main()</code>	1) <code>database.connect()</code> — 2) levanta Express en <code>/api</code> — 3) <code>whatsappService.initialize()</code> (pide QR si hace falta) — 4) <code>cronJobs.init()</code>

Database clase src/database/index.js

Encapsula todo el acceso a Supabase (Postgres + Storage). Es el único lugar del código que sabe cómo están armadas las tablas `clientes`, `cuotas` y `pagos`.

MÉTODO	QUÉ HACE
<code>connect()</code>	Crea el cliente de Supabase con las credenciales del <code>.env</code> y valida que la tabla <code>clientes</code> exista (si no corriste <code>schema.sql</code> , falla acá con un mensaje claro).
<code>crearVenta({nombre, telefono, producto, cuotas, valorCuota})</code>	Inserta el cliente y genera automáticamente todas sus cuotas (una por mes, a partir de hoy). Usada por <code>POST /api/ventas</code> .
<code>getClientByTelefono(telefono)</code>	Busca un cliente por número (normalizado). Es cómo el bot identifica

	quién le está escribiendo.
<code>getProximaCuota(clienteId)</code>	Trae la cuota pendiente más próxima de un cliente.
<code>getCuentasPendientes(clienteId)</code>	Lista todas las cuotas sin pagar de un cliente.
<code>subirComprobante(clienteId, base64Image)</code>	Sube la foto del comprobante al bucket <code>comprobantes</code> de Supabase Storage y devuelve la ruta guardada.
<code>registrarPago(pago)</code>	Sube el comprobante (si viene), inserta el pago y marca la cuota como <code>pagado</code> .
<code>getPagosDelDia(fecha)</code>	Trae todos los pagos registrados en un día, con nombre del cliente y número de cuota incluidos (join).
<code>getClientesAtrasados()</code>	Lista clientes con al menos una cuota vencida y sin pagar.
<code>getCuentasAtrasadas(clienteId)</code>	Cuentas vencidas de un cliente puntual (se usa para decidir si pasa a estado "moroso").
<code>actualizarEstadoCliente(clienteId, estado)</code>	Cambia el estado del cliente (<code>aldia</code> / <code>moroso</code>).

WhatsAppService `clase` `src/whatsapp/index.js`

Maneja la conexión con WhatsApp (vía `whatsapp-web.js`, que emula WhatsApp Web) y decide qué hacer con cada mensaje entrante.

MÉTODO	QUÉ HACE
<code>initialize()</code>	Crea una sesión de WhatsApp por cada valor de <code>WHATSAPP_SESSIONS</code> . Si no hay sesión guardada, imprime el QR en la terminal y lo guarda como imagen PNG. Registra los listeners de eventos (mensaje recibido, conectado, desconectado, cambio de estado).
<code>resolverNumero(message)</code>	WhatsApp a veces oculta el número real de un contacto que no está en la agenda detrás de un id opaco (<code>@lid</code>) por privacidad. Este método intenta recuperar el número real vía <code>message.getContact()</code> ; si no puede, devuelve <code>null</code> y el mensaje se ignora.
<code>handleMessage(message, client)</code>	Punto central de decisión: ignora grupos/estados, resuelve el número real, y según el tipo de mensaje delega en <code>pagosService.procesarComprobante()</code> (si es una imagen) o <code>iaService.procesarMensaje()</code> (si es texto). Loggea todo ( recibido,  respuesta,  error) para poder debuggear.
<code>enviarMensaje(numero, mensaje)</code>	Envío saliente (usado por los cron jobs): normaliza el número y lo manda por una de las sesiones conectadas.

Nota técnica: el número de teléfono que se usa para *buscar al cliente en la base* (resuelto vía `resolverNumero`) y el *id de chat al que se responde* (`message.from`, que puede ser `@lid`) se manejan por separado — por eso `procesarComprobante` recibe ambos valores.

IAService `clase` `src/services/ia.service.js`

Toda la interacción con OpenAI: entender qué quiere el cliente y leer los comprobantes.

MÉTODO	QUÉ HACE
<code>procesarMensaje(mensaje, numeroCliente)</code>	Punto de entrada para mensajes de texto. Si el número no está registrado, pide identificación. Si está, clasifica la intención y responde según corresponda.
<code>detectarIntencion(mensaje)</code>	Le pregunta a GPT-3.5 si el mensaje es <code>consultar_deuda</code> , <code>reportar_pago</code> o <code>duda</code> . Es tolerante a respuestas ruidosas del modelo (usa <code>.includes()</code> en vez de comparación exacta).
<code>consultarDeuda(cliente)</code>	Arma el mensaje con la próxima cuota, fecha de vencimiento y días de atraso si corresponde.
<code>calcularAtraso(fechaVencimiento)</code>	Diferencia en días entre hoy y la fecha de vencimiento.
<code>responderDuda(mensaje, cliente)</code>	Respuesta libre de GPT-3.5 para preguntas generales (con un prompt de sistema que lo mantiene en personaje).
<code>extraerDatosComprobante(texto)</code>	Le pasa el texto que sacó el OCR a GPT-3.5 y le pide que devuelva JSON con <code>monto</code> y <code>nombre</code> del pagador. Es más robusto que una regex porque tolera errores de lectura del OCR.

OCRService `clase` `src/services/ocr.service.js`

Convierte la imagen del comprobante en texto plano.

MÉTODO	QUÉ HACE
<code>leerImagen(base64Image)</code>	Usa Tesseract.js (idioma portugués) para extraer el texto de la foto del comprobante.
<code>leerImagenGoogleVision(base64Image)</code>	Alternativa con Google Cloud Vision para mejor precisión — no está en uso todavía, requiere instalar <code>@google-cloud/vision</code> y configurar credenciales.

PagosService `clase` `src/services/pagos.service.js`

Orquesta el ciclo completo de un pago: desde que llega la foto hasta que se confirma o se pide aclaración. Es el "cerebro" del flujo de cobranza.

MÉTODO	QUÉ HACE
<code>procesarComprobante(media, numeroCliente, chatId, client)</code>	1) OCR → texto — 2) IA extrae monto/nombre — 3) busca al cliente — 4) compara el monto contra la cuota

pendiente — 5) si coincide, registra el pago y confirma (avisando si era la última cuota); si no coincide, pide confirmación manual.

`extraerDatosPago(texto)`

Intenta primero con `iaService.extraerDatosComprobante()` ; si falla o no encuentra nada, cae a una regex simple como respaldo.

`verificarAtrasos(clienteId)`

Después de cada pago, revisa si quedan cuotas vencidas y actualiza el estado del cliente (`aldia` / `moroso`).

`generarInformeDiario()`

Arma el texto del resumen diario (pagos recibidos + pendientes + total) con los números reales de la base — sin usar IA, para que las cifras nunca se inventen.

CronJobs `clase` `src/jobs/cron.jobs.js`

Las tareas que corren solas, sin que nadie las dispare.

MÉTODO	CUÁNDO CORRE	QUÉ HACE
<code>enviarRecordatorios()</code>	Todos los días 9 AM	Le escribe a cada cliente atrasado recordándole su próxima cuota.
<code>generarInformeDiario()</code>	Todos los días 9 PM	Te manda a vos (<code>ADMIN_WHATSAPP</code>) el resumen del día armado por <code>pagosService</code> .
<code>verificarPagosPendientes()</code>	Cada hora	Placeholder — todavía no tiene lógica real implementada.

API REST `módulo` `src/routes/index.js`

Rutas Express montadas bajo `/api` , pensadas para un futuro panel de administración (o para cargar ventas sin depender de WhatsApp).

ENDPOINT	QUÉ HACE
<code>POST /api/ventas</code>	Registra un cliente nuevo y sus cuotas (llama a <code>database.crearVenta()</code>).
<code>GET /api/dashboard</code>	Devuelve pagos del día, clientes atrasados y el total recibido — pensado para alimentar el panel React.

`normalizarTelefono(numero)` `utilidad` `src/utils/telefono.js`

Función de una línea, pero crítica: saca todo lo que no sea dígito de un número de teléfono. Se usa en *todos* los puntos donde se guarda o se busca un teléfono, para que `+57 317 277 8722` , `573172778722@c.us` y `573172778722` se traten siempre como el mismo número.